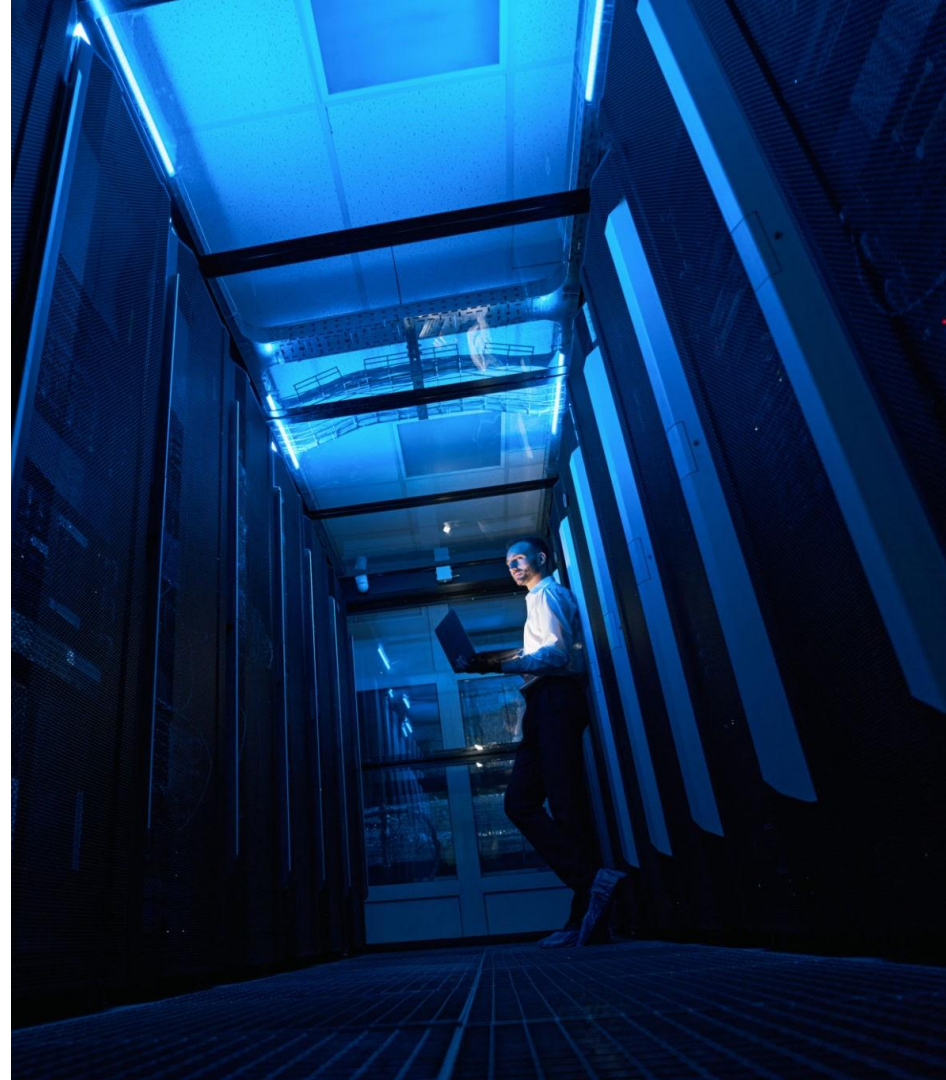


Designing Observable Services

Microservices Patterns - Chapter 11.3

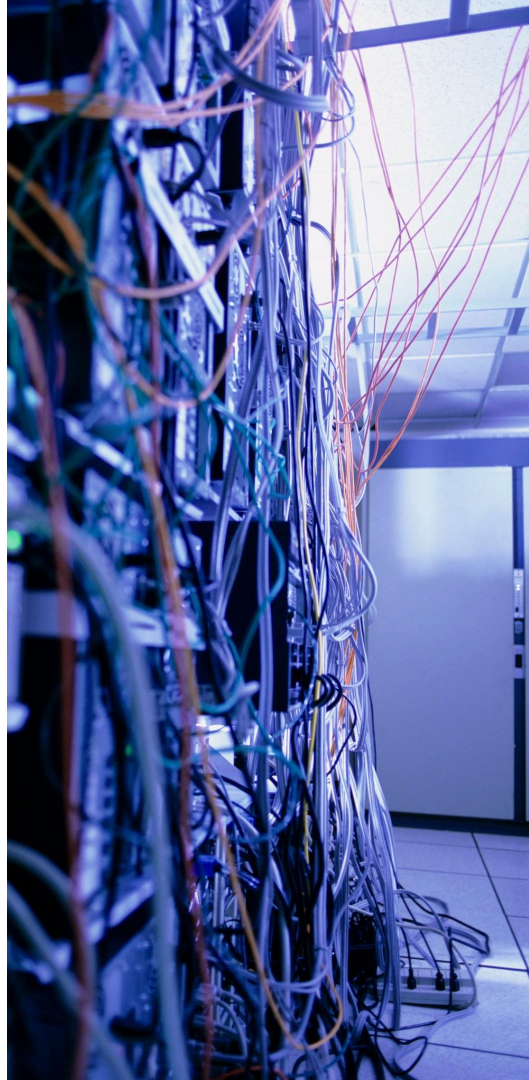
Hüsamettin IŞIKTAŞ
25501005

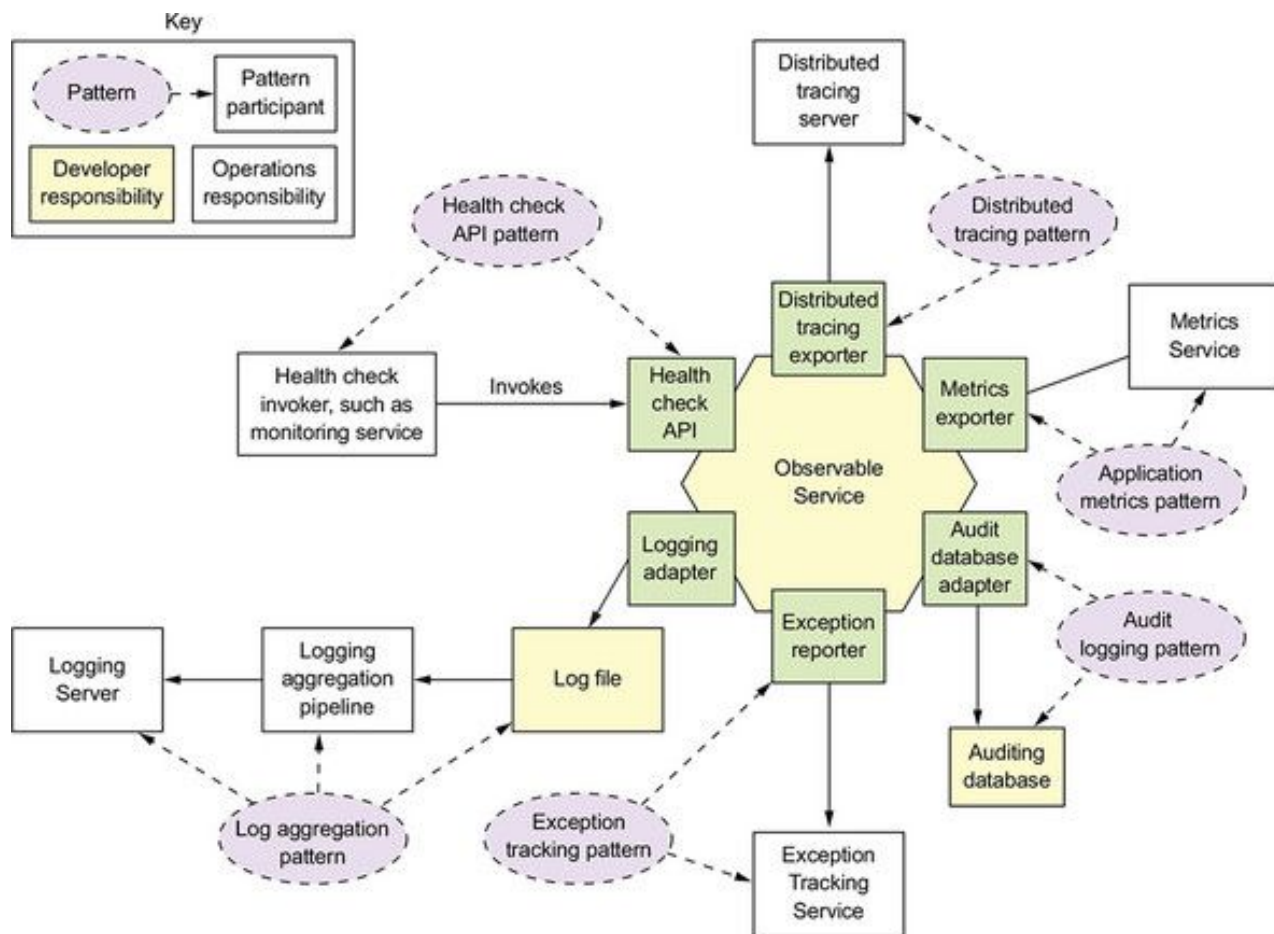


Why Observability?

As systems become more complex, maintaining them becomes more difficult. Therefore, we need systems and services to be observable. To create observable services, the following are 6 patterns that services should follow.

1. **Health Check API**
2. **Log Aggregation**
3. **Distributed Tracing**
4. **Application Metrics**
5. **Exception Tracking**
6. **Audit Logging**





1. Health Check API Pattern

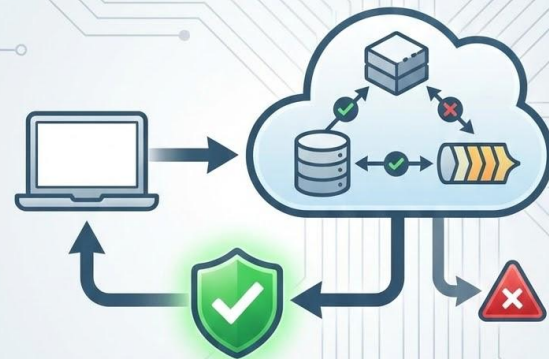
A service might be running but unable to handle requests.

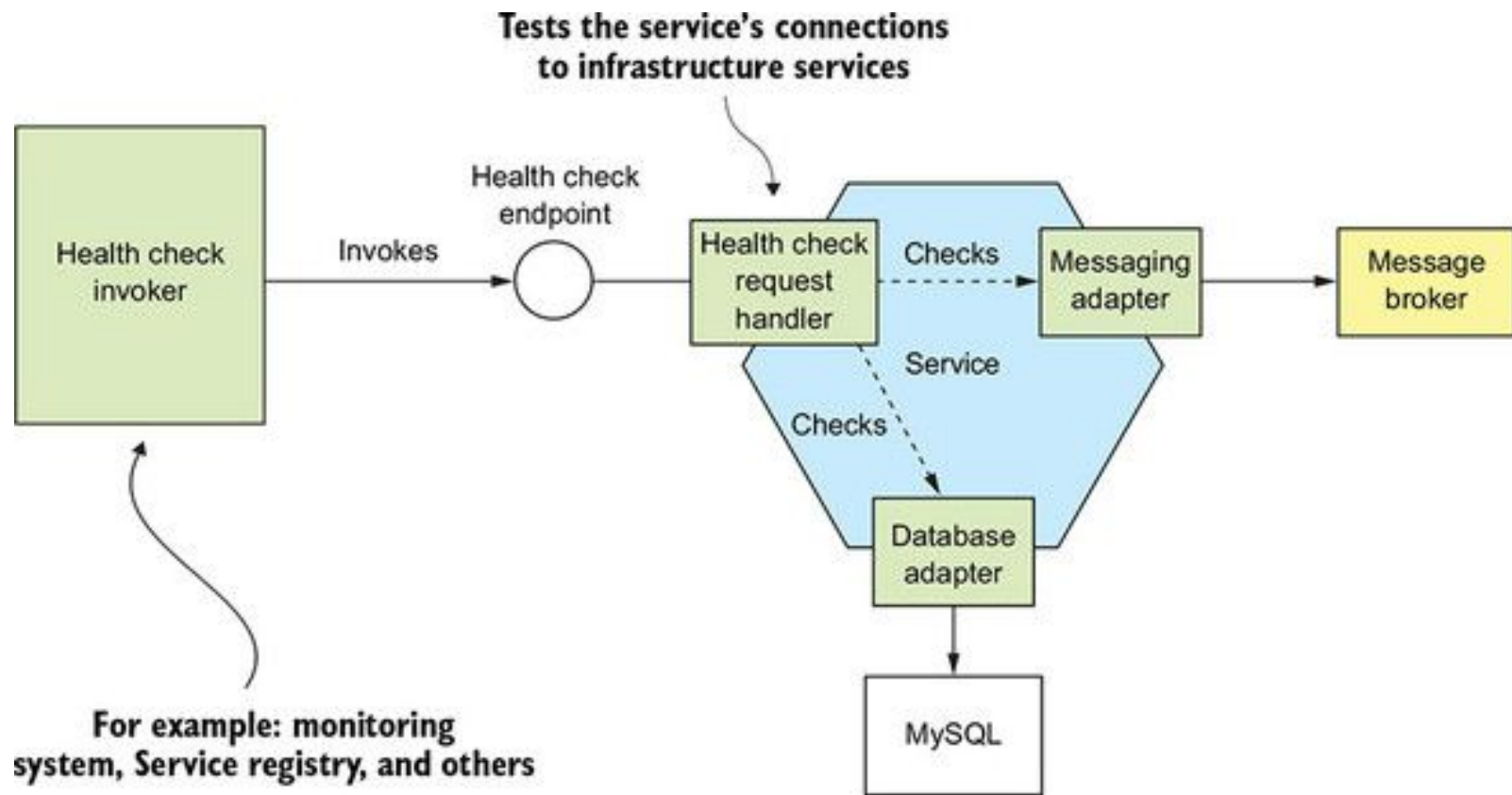
Common Issues:

- Database connection failures.
- Deadlocks.
- Still warming up (not ready).

Implementing Health Checks:

- **Implementation** - Development
- **Configuration** - Operations





2. Log Aggregation Pattern

A single request may span between multiple services and handling the logs turns into nightmare. We can't SSH to 50 services but we can centralize logs.

Developer Responsibilities:

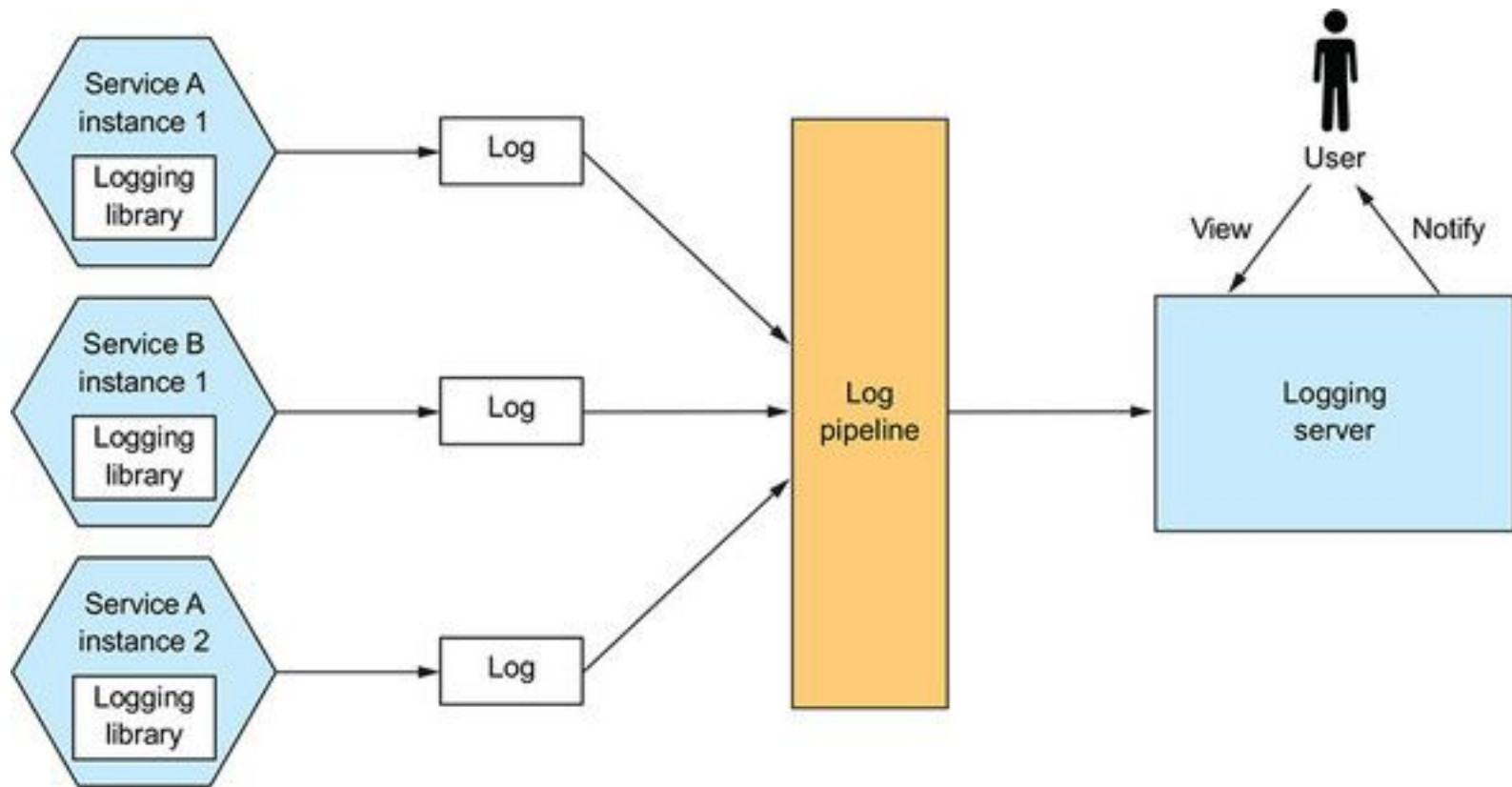
- Which library to use or write own library? (Logback, Log4j, JUL)
- Where to save logs? (Known file, stdout)

Operations Responsibilities:

- Collect
- Save
- Search (ELK Stack, Fluentd, Apache Flume)

! Log aggregation is also important for debugging.





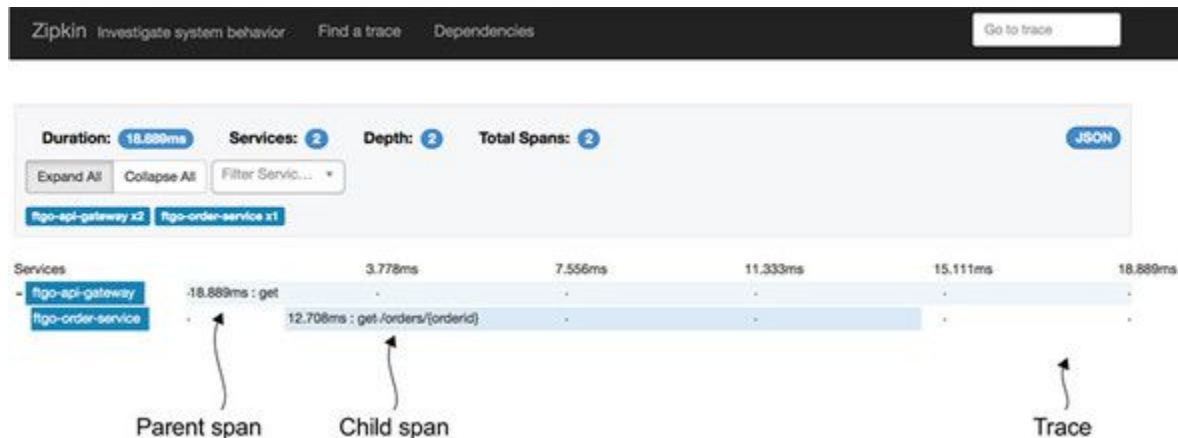
3. Distributed Tracing Pattern

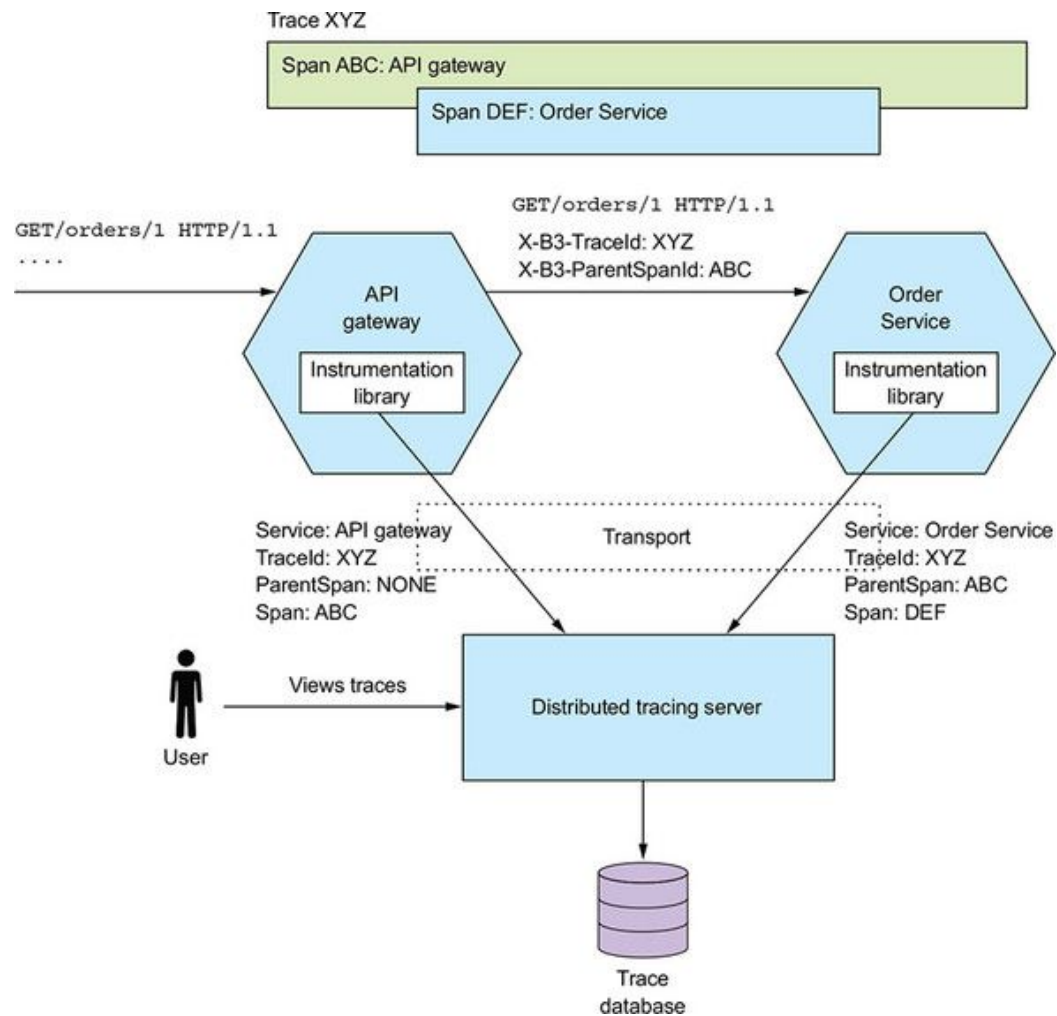
Our system might be running slower than expected. And finding which services are causing the system-wide slowdown is a task that becomes more difficult to backtrace as the system becomes more complex.

```
2018-03-04 17:38:12.032 DEBUG
```

```
[ftgo-order-service,8d8fdc37be104cc6,8d8fdc37be104cc6,false]
```

```
7 --- [...Truncated...]
```





4. Application Metrics Pattern

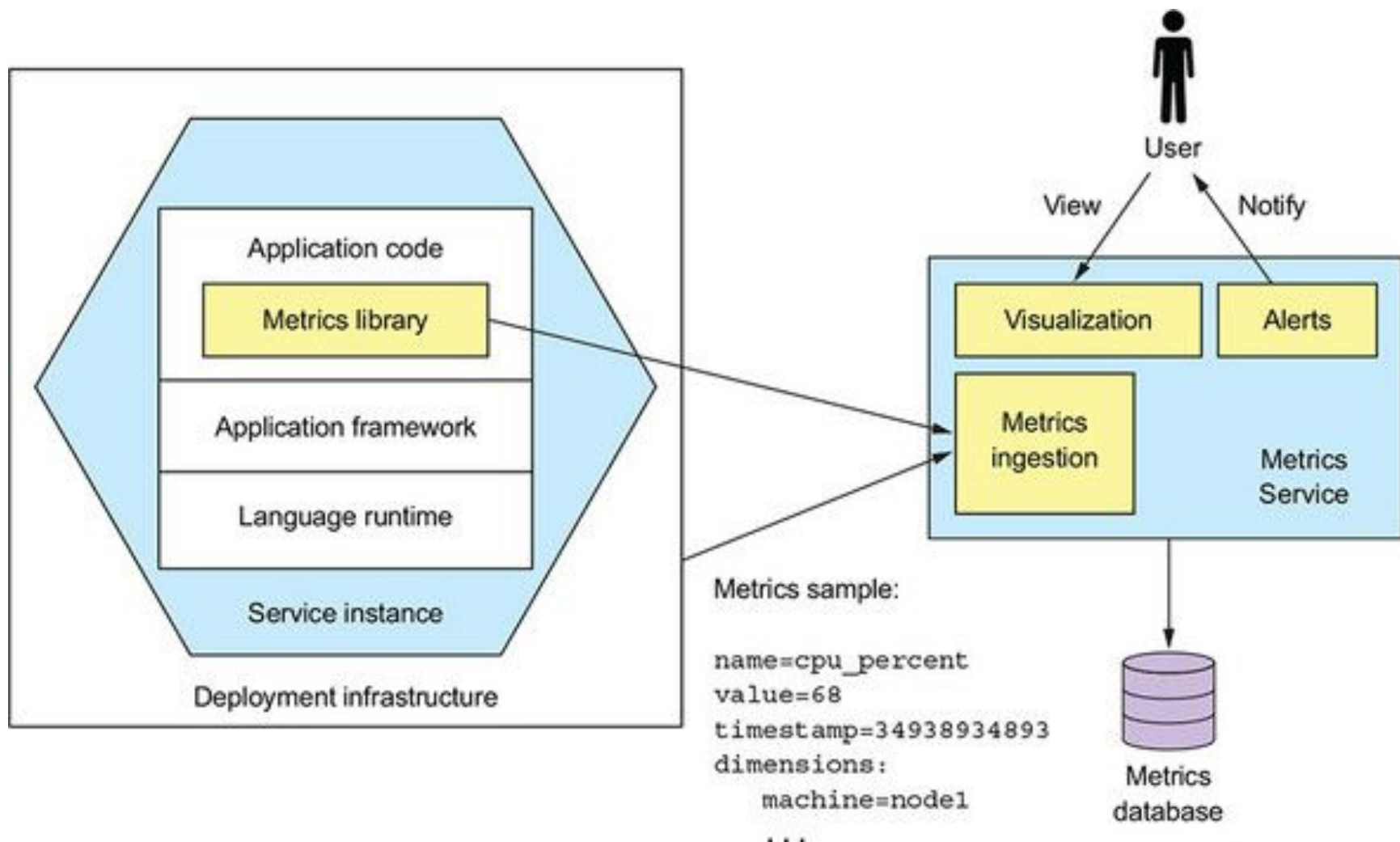
We monitor how our system works through metrics. Metrics can represent the **infrastructure** (CPU, RAM, Disk etc.), or **service-specific** metrics can be created (order count, fail, success etc.).

Metrics are:

- Sampled **periodically**
- Generally includes **name, value, and timestamp** attributes
- Data is sent to the metrics service using **push** or **pull** methods

! Prometheus and **Grafana** are the most known tools.

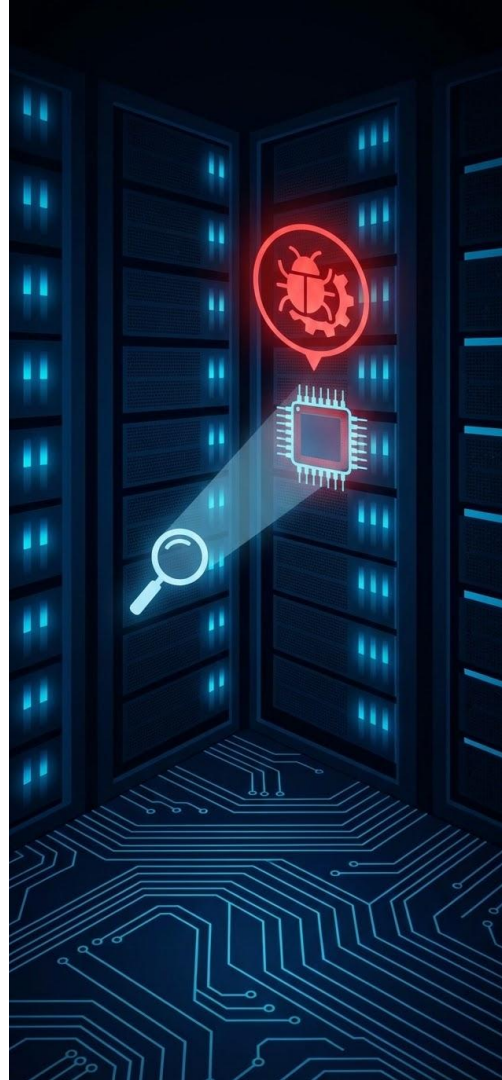


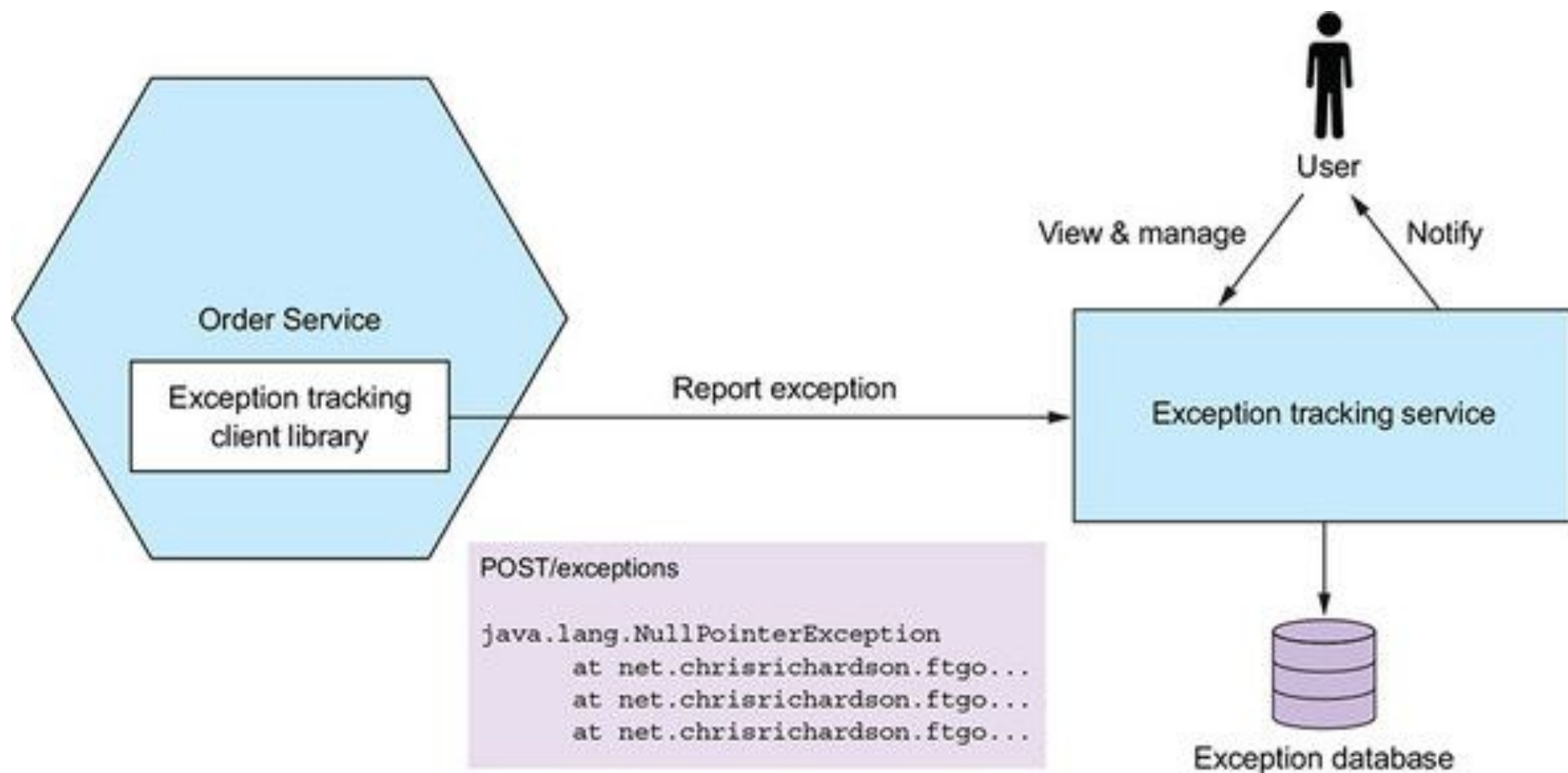


5. Exception Tracking Pattern

A service should rarely log exceptions, and when it does, identifying the source of the exception is crucial. We can do this by getting alerts from the logging section using the classic method, but the drawbacks are:

- **Logs are** recorded as **single lines**, but exceptions may contain details requiring **multiple lines**.
- There is no mechanism to change only the **detail level** of the exception without changing the detail level of **all logs**; the exception must be **manually** searched in the issue tracker.
- There is also no mechanism to consolidate **duplicate errors into a single point**.



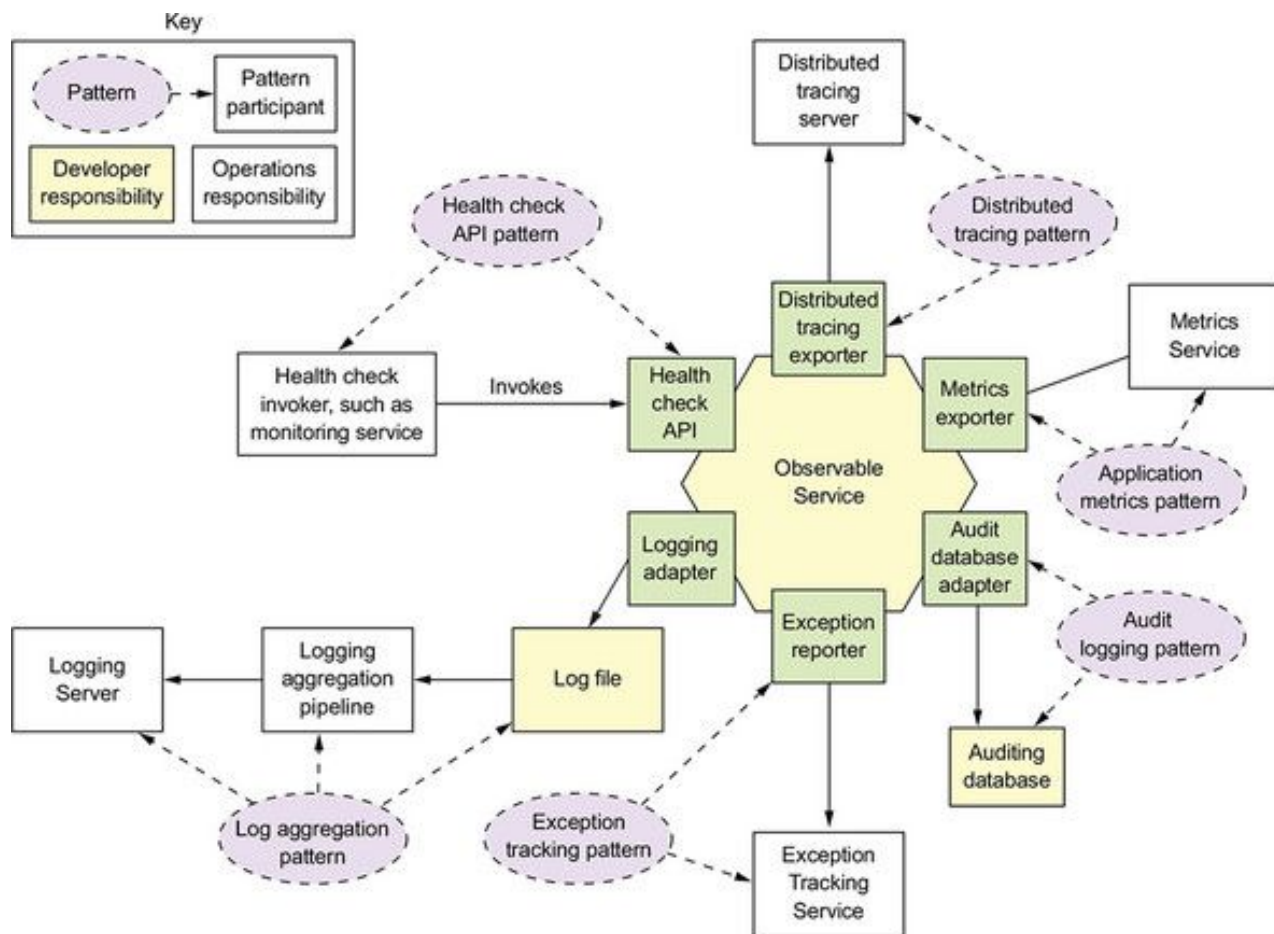


6. Audit Logging Pattern

The purpose of audit logging is to record each **user's actions**. An audit log is typically used to help **customer support**, **ensure compliance**, and **detect suspicious behavior**. Each audit log entry records the identity of the user, the action they performed, and the business objects. An application **usually** stores the audit log in a **database** table.

There are a few different ways to **implement audit logging**:

- Add audit logging code to the business logic.
- Use aspect-oriented programming (AOP).
- Use event sourcing.



**Thanks for
listening.**